UI Sharing
Compose Multiplatform

Contents

- **Introduction**
- **What is Compose Multiplatform?**
  - Advantages
  - Disadvantages
- **Setup**
  - Install Homebrew
  - Install doctor
  - Install Kotlin Multiplatform
- **Create Compose Multiplatform Project**
  - App Structure
  - Setup HTTP client
    - Adding the dependencies
    - Create the models
    - Create Ktor client
    - Initialize Ktor
  - Integrate Koin
  - Add fonts
  - Add drawables
  - Building the HomeScreen
  - Building the AgentDetailsScreen
  - Navigation

Introduction

In this book, we will build an app using **Compose Multiplatform**. The API that we will use is called **Valorant-API,** and we will display the **Valorant** characters and information about them**.**

What is Compose Multiplatform?

Compose Multiplatform is a declarative framework for sharing UIs across multiple platforms. It is based on **Kotlin Multiplatform** and **Jetpack Compose**.

Advantages

1. Multi-platform support

Android, iOS, Desktop, and Web

2. Cross-Platform Reach

You can target a wider audience by deploying your app on multiple platforms with a single codebase.

3. Learning Curve

If you're familiar with **Jetpack Compose** the transition to **Compose Multiplatform** can be relatively smooth due to the shared concepts and APIs.

Disadvantages

1. Maturity

It is still a relatively new technology. Some features of platform-specific capabilities might be less mature compared to native development or other frameworks like **Flutter**.

2. Ecosystem

The ecosystem around **Compose Multiplatform** is still growing. While there are a growing number of libraries and tools, they might not be as extensive as native development on specific platforms.

Setup

We must ensure we have everything set up: **Android Studio**, **Xcode**, and **Java**.

To check if these things are properly set up, we will use a tool called **kdoctor**. To install it we need to have **Homebrew** installed.

Install Homebrew

If you do not have **Homebrew** installed, run this command in your **Terminal.**

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/inst
all.sh)"
```

Install kdoctor

Now with this installed, we can run this command to install **kdoctor**.

```
brew install kdoctor
```

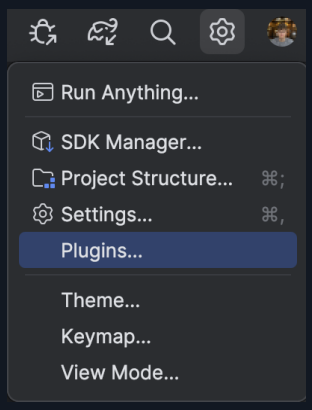
After the installation is completed, run **kdoctor** in the console.

```
kdoctor
```

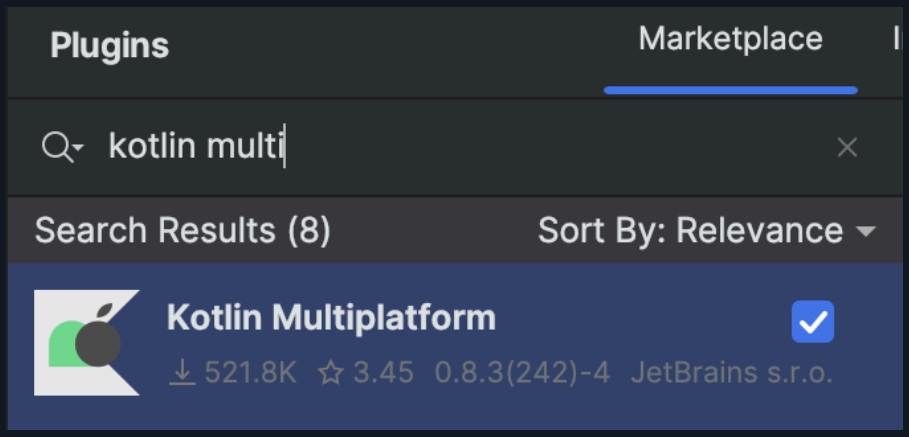
If something has a **[x]** you can check **this list** to see how to fix the problem.

Install Kotlin Multiplatform Plugin

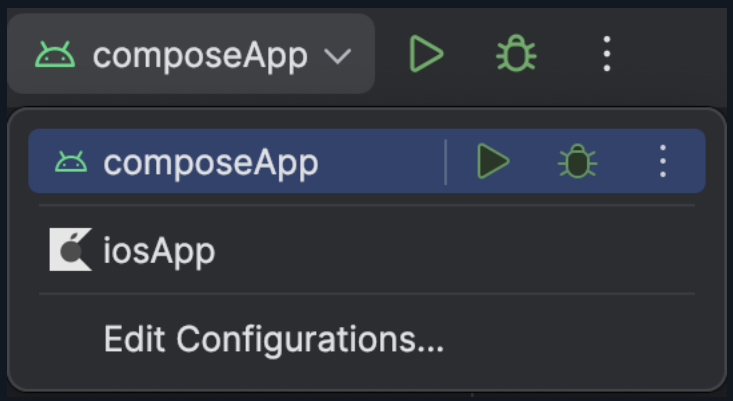
Before creating the **CMP project** we need to do one more thing in the **Android Studio**. To run the **iOS** app from **Android Studio** we need to install the **Kotlin Multiplatform** plugin. For that, open the **plugins** tab from the top-right corner.



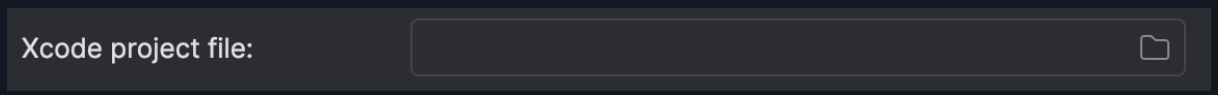
Go to the **Marketplace** search for the plugin and install it. After installing, restart the **Android Studio** to enable the plugin.



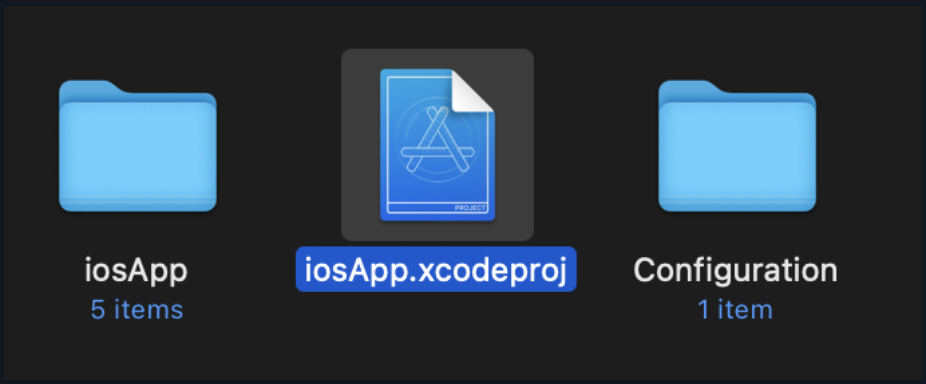
Then, open the run destinations and select **“Edit configurations”.**



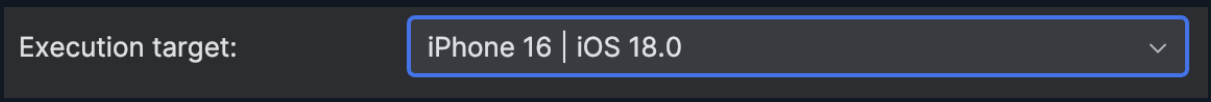
Now we need to set up an iOS emulator. So, click on iOS Application and click on the **“Add new run configuration”.**



For the **Xcode project file**, you must select the `iosApp.xcodeproj` from the **iosApp** package. Then the rest of the fields will autocomplete.



At the **Execution target** field, pick your desired iOS device. In my case, I use an **iPhone 16** with iOS 18.0.



Thanks for reading! To learn more about building an app using **Compose Multiplatform**, make sure to get the full e-book with a **20% discount** until **October 21st**.